KCWI Blue+Red Reduction Workflow

Nov 2, 2026 · @Yuanze

Overview

Frames are reduced one at a time, aligned blue and red together with `kcwi_xalign`, stacked per channel on one shared grid, checked against each other, and, if pPXF-fitted maps are wanted, handed to GIST. The change from the old workflow: no point source and no per-channel `kcwi_align` / `kcwi_astrometry`; the absolute position goes into the shift lists before stacking.

Blue and red are aligned together, stacked apart, and checked together



KCWI blue+red reduction flow · 6 stages, 1 check

The only step that sees both channels at once is `kcwi_xalign`; if its `wcs` check fails, fix the `.par` files or rerun the shifts, never patch the stacked cubes.

Primary references:

The KCWIKit `https://github.com/yuanzeding/kcwi` (dev branch)

KCWI_DRP, python 3.11 version 1.1.1, make sure you have 'keckdrpframework~=0.1.dev0'
`https://github.com/yuanzeding/KCWI_DRP/tree/old-state` (old-state branch)

KSkyWizard, version 0.2.2025-06-04

`https://github.com/zhuyunz/KSkyWizard/tree/auto` (auto branch)

Software and where it lives

Four code bases do the work: the KCWI DRP for single frames, KSkyWizard for red-channel sky, KcwiKit for alignment and stacking, and, optionally, GIST downstream for pPXF fits.

Tool	Does	Location
KCWI_DRP	Per-frame reduction to `_icubes`, `_vcubes`, `_mcubes`, `_ecubes`; extra cosmic-ray pass (`extra_CRR`)	GitHub repo of Yuanze: `KCWI_DRP`, branch `oldstate` (includes the bcrmsk patch to `kcwidrp/primitives/RemoveCosmicRays.py`)
KSkyWizard	Red-channel sky subtraction (outputs in `ksky/`, `kr*_zap`)	KSkyWizard package (separate install) https://github.com/zhuyunz/KSkyWizard/tree/auto
`kcwi_automask.py`	Region masks for sources, used by the DRP and the stacker	`https://github.com/yuanzeding/kcwi` dev branch
`kcwi_check_saturation.py`	Saturation screen on legacy QSO frames	stand-alone script (ask Yuanze if needed)

Tool	Does	Location
`kcwi_xalign.py`	Blue+red shift lists by cross-correlation; `wcs` step checks the stacks. Designed for extended and point sources.	Part of `KcwiKit` `KcwiKit/kcwikit/scripts/`
`kcwi_shiftcheck.py`	Measures how wrong a shift list is; compares two	`KcwiKit/kcwikit/scripts/`
`kcwi_stack()`	Drizzles frames onto the output grid, `crr_split` cosmic-ray rejection	`KcwiKit/kcwikit/kcwi/kcwi.py`
`kcwi_align()`, `kcwi_astrometry()`	Deprecated: old per-channel alignment and point-source astrometry, replaced by `kcwi_xalign`	`KcwiKit/kcwikit/kcwi/kcwi.py`

Frames live in one folder per observing night: DRP output in `redux/` (blue `kb*` frames) and sky-subtracted red frames in `ksky/` (`kr*_zap`). Each target's stacks, with its `.list` and `.par` files, go in a separate stacking folder. `kcwi_xalign` needs either `Montage` (significantly faster, **DO NOT USE** MontagePy; bugs confirmed) or the `reproject` package.

Step 1 — Single frames: DRP, saturation, sky

Each exposure is reduced on its own; the products are cubes in sky coordinates from the header WCS, which can be off by up to ~1″ (usually less). In general, you need the following procedures:

1. **DRP.** Run KCWI_DRP with the `.cfg` file provided separately. It turns the DRP's wavelength corrections off (`radial_velocity_correction = "none"`, `air_to_vacuum = False`), because the `WavelengthCorrections` step still has problems. Keep `extra_CRR` on. Users should decide whether they want a cosmic ray mask (`<frame>_intf_bcrmsk.fits`) which exclude bright continuum source from CR rejection. This is a problem for almost all bright sources observed by KCWI. The extra

cosmic-ray pass now reads `<frame>_intf_bcrmsk.fits` and passes it as `inmask`, so masked source pixels are not called cosmic rays. Build the region masks with `kcwi_automask.py` (`-wl` uses polygon source contours; `--all` makes one region per exposure).

2. **Saturation (bright, point-like source).** Screen the raw frames with `kcwi_check_saturation.py`. The Analog to Digital Converted has a 16-bit limit at 65535 DN; blue clips at ~8,700–20,600 e^-, below full well, so the clip is only clean on the raw frame. A frame is saturated when a railed group is ⩾ 4 pixels along wavelength, ⩽ 3 columns wide and ⩾ 5 pixels. Saturated frames must stay out of any PSF-subtraction stack but can still serve cosmic-ray rejection.
3. **Red sky.** Sky-subtract the red frames with KSkyWizard; the stack lists then point to `ksky/kr<date>_<nnnnn>_zap`. Blue frames are taken from `redux/kb<date>_<nnnnn>`.

How to run KCWI DRP?

Follow the KcwiKit docs (`kcwikit/docs/Running_blue_DRP.md`, `Running_red_DRP.md`). Between stages, delete the science rows (`OBJECT`, and `SKY` for blue) from `kcwib.proc` / `kcwir.proc` so the DRP redoes them. Masks that the docs **draw by hand in DS9 are now made by** `kcwi_automask.py` (modes in the masks paragraph below). Here are some brief descriptions.

Blue channel (`reduce_kcwi -b -f kb*.fits -g -c <cfg>`):

1. `-st 1`: run until after sky subtraction.
2. Sky masks: `kcwi_automask.py kb*_intf.fits --smsk --plots` writes diagnostic plots, `<base>.reg` and `<base>_smsk.fits`. Delete the science rows and the frames' `*_sky.fits` in `redux/`, then `-st 2` to cube creation.
3. Cosmic-ray false positives on bright sources (FLAGS > 3 around a QSO in the `icube`): `kcwi_automask.py kb*_intf.fits --bcrmsk` writes `<base>_intf_bcrmsk.fits`; set `bcrmsk = True` in the `.cfg` and rerun.
4. Optional median filtering: KCWI Flat-fielding does not completely remove inhomogeneity, so one side of the CCD is always brighter than the other. Run this when the uneven illumination pattern is problematic for your science.
   `kcwi_collapse kb*_icube.fits`, `kcwi_flatten_cube kb*_icube.fits`, masks with `kcwi_automask.py kb*_icube.thum.fits --smsk`, then `kcwi_medfilter kb*_icube.fits`. Required, not optional, for bright point sources because of the diffraction spikes.

5. Delete the science rows in `kcwib.proc` / `kcwir.proc`, `-st 3` to the end (flux calibration).
6. Run full reduction (do not add `-st`) on standard stars. Trim the wavelength range and mask prominent absorption line.
7. Optional: combine several standard stars with `kcwi_combinestd <name>.list`, replace an `INVSENS` row in the proc table with the combined file, remove the rest of them and delete the science rows and rerun `-st 3`.

Red channel (`reduce_kcwi -r -f kr*.fits -g -c <cfg>`):

1. `-st ff`: run until after `_intf.fits`.
2. Cosmic-ray masks: `kcwi_gen_log <night>.log -f kr*.fits`, then `kcrm_group_frames <night>.log kcrm_<night>.json -c -d -i`; remove standard stars from the JSON and fix the grouping; `kcrm_create_crmsk kcrm_<night>.json` (`-s <sigma>`, default 3) writes `*crmsk.fits`. For bright point sources inspect the masks: you may create a `kr<date>_<frame>_crmsk_recover.reg` region un-masks false positives.
3. Delete the science rows, `-st 2` to cube creation.
4. Median filtering, as for blue (thumbnail masks: `kcwi_automask.py --json kcrm_<night>.json --thum`).
5. Delete the science rows, `-st 3` to the end.
6. KSkyWizard: telluric correction (optionally combine `*_invsens_updated.fits` with `kcwi_combinestd` and load it in the GUI), then PCA sky subtraction. Source masks for it: `kcwi_automask.py kr*_wlimg.fits` writes the `<base>.reg` the GUI reads.

Masks. `kcwi_automask.py` has four modes, each writing a DS9 region file you can check and edit, then rebuild the FITS mask with `--convert`:

Mode	Input	Output	Used by
intf (default)	`_intf.fits`	`<base>.reg`, `_smsk.fits` with `--smsk`	DRP sky subtraction
`--bcrmsk`	`_intf.fits`	`_intf_bcrmsk.fits`	DRP cosmic-ray pass (`inmask`)
`--thum`	`_icube.thum.fits`	`_icube.thum.reg`, `_icube.mask.fits` with `--smsk`	`kcwi_medfilter`

Mode	Input	Output	Used by
`--wl`	`_wlimg.fits`	`<base>.reg`	KSkyWizard / ZAP

It finds continuum sources only: a source seen only in an emission line still needs a hand-drawn region. `--json <file> [--all]` takes the frames from a grouping JSON; `--no-clobber` keeps hand-edited regions; `--plot` writes a check PNG per frame.

Each frame gives `<base>_icubes.fits` (flux), `_vcubes` (variance), `_mcubes` (mask) and `_ecubes` (exposure).

Step 2 — The stack files

Each channel has its own set of text files, named after the target; the red ones carry `-red` (`IRAS03582+6012.list` , `IRAS03582+6012-red.list`). Run every KcwiKit step in the folder that holds them.

File	Contents	Written by
`<t>.list`	One frame per line: base path (no `_icubes.fits`) and two trim values, e.g. `.../redux/kb241008_00078 3 3`	you
`<t>.par`	Grid and alignment settings, `key value` per line (below)	you
`<t>.shift.list`	Frame name and CRPIX shift (x, y) in frame pixels	`kcwi_xalign shifts` (old: `kcwi_align`)
`<t>.preshift.list`	Optional coarse offsets added to CRVAL before the shifts [arcsec]	you
`<t>.astrom.list`	One absolute correction (dRA, dDec) [arcsec] for the whole channel	old `kcwi_astrometry`
`<t>.flx.list`	Optional flux weights	`kcwi_check_flux`

The .par keys that matter for stacking. Both channels must share the same grid:

```
dimension 145 145
xpix 0.146
ypix 0.146
orientation 0
stack_ad 60.6377 60.3452
```

`stack_ad` is the RA, Dec of the grid centre in degrees. Without it the grid is centred on the first frame's header position, so it moves with the list and differs between channels. `wavebin`, `align_box`, `ref_xy`, `ref_ad` and `ref_fn` are only read by the old tools, now deprecated, `kcwi_align` / `kcwi_astrometry`.

Sign conventions. `kcwi_stack` places frame k by `CRPIX += shift_k` after `CRVAL += preshift/3600`. An `astrom.list` value is applied as `CRVAL += value/3600`, with dRA in RA coordinate (not on-sky) arcsec: IRAS03582's 17.05″ is 8.4″ on the sky at Dec 60°.

Step 3 — Align blue and red together: kcwi_xalign

`kcwi_xalign shifts` writes both channels' `.shift.list` from the data, with no point source, and puts blue and red on the same sky to about 0.01–0.02″. It replaces `kcwi_align` plus a separate `kcwi_astrometry` per channel.

```
python kcwi_xalign.py shifts <t>.list <t>-red.list --z <z> [absolute option]
```

How it measures.

- A white-light image of every frame, 400 Å on each side of the dichroic (`--wb`, `--wr` to change), with emission lines (at the target's redshift) and sky lines (observed frame) cut out by `--z <redshift>`, all projected onto one 0.1″ grid (`--gridpix`).
- Within a channel each frame is fitted to the mean of the others, frame = a · template(x − d) + b + gradient. The exact leave-one-out update is iterated until no frame moves by more than 0.003 px; the channel mean is the gauge.
- The red template is fitted to the blue one: one blue–red offset. Frames at different position angles work; `--highpass FWHM` removes large-scale background differences.
- Search radius `--radius` (default 2″); old lists are backed up as `.shift.list.bak_<time>`, and a JSON solution and PNG go to `kcwi_xalign/`.

The absolute position — choose one. Without any of these the stacks sit at the mean blue header position, which is usually good to < 1″.

Option	Use when	Notes
`--ref image.fits`	An external image exists (HST, Legacy Survey, Pan-STARRS)	`--ref-radius` 15″, `--ref-smooth` to match the PSF
`--astrom <t>.astrom.list`	A `kcwi_astrometry` correction exists from an old stack and you have checked it against a reference image	Use the BLUE file for both channels; tied to blue frame 0 (≲ 0.02″ on synthetic data)
`--abs-offset dRA dDec`	You know the correction by hand	Same convention as `astrom.list`

Checking old or new shift lists (optional). `kcwi_shiftcheck.py` places every frame as `kcwi_stack` would and measures each frame against the others, plus blue against red:

```
python kcwi_shiftcheck.py <t>.list <t>-red.list --a '.shift.list.bak_<time>'
--b .shift.list --z <z> --plot check.png
```

Residuals are relative to each channel's median frame; "!" marks more than 50 mas. On Mrk248 the old `kcwi_align` lists had two red frames 0.44″ off and a 200 mas blue–red offset; the new lists agree to < 1 mas within a channel and 5 mas between channels.

Step 4 — Stack: kcwi_stack with crr_split

crr_split added in 2026 Oct has vastly improved the CR rejection, effectively removing all CR residuals above ~2sigma when 7 frames are combined, so it is recommended to be turned on.

Stack both channels with the new shift lists, `use_astrom=False` (the absolute offset is already in the shifts) and the same `stack_ad` in both `.par` files.

```
from kcwikit.kcwi.kcwi import kcwi_stack
for lst in ['<t>.list', '<t>-red.list']:
    kcwi_stack(lst, crr_split=True, crr_split_mode='frame',   # 'frame' for
<= 7 frames
               crr_save_mask=True, overwrite=True)
```

Outputs: `<t>_icubes.fits`, `_vcubes`, `_mcubes`, `_ecubes`, and `kcwi_stack/<t>.crrsplit.fits` (rejections per voxel, plus a table per frame).

How crr_split rejects cosmic rays. After the frames are on the common grid, each frame (or group) is compared with the others. A voxel is masked in that frame only if its excess is > `crr_nsig` = 5σ and > `crr_rel` = 0.5 × the local source level. Before the test each frame is scaled to the others along wavelength (`crr_zratio` = 51), a 15-pixel running median is removed (`crr_zmed`), and a gradient term (`crr_fgrad` = 0.5) protects QSO flanks from sub-pixel misregistration. Flags are grown into connected neighbours.

Few frames. `crr_split_mode='auto'` makes two groups for anything up to 17 frames, which dilutes a cosmic ray in its own group. Thresholds in units of one frame's σ:

Frames	Mode	Caught above	And above
2	either (each frame vs the other)	~7σ	0.5 × source
3	auto, frames 0 and 1	~12σ	1 × source
3	frame	~6σ	0.5 × source
4	auto (2 + 2)	~10σ	1 × source
4	frame	~6σ	0.5 × source

With 2 frames the high one is masked, so a real difference (sky residual, seeing) looks the same as a cosmic ray. Voxels covered by one frame are never tested.

Reusing masks. `crr_save_mask=True` writes `<base>.crmsk.fits` per frame; `crr_reuse_mask=True` loads them instead of recomputing, e.g. to reject with all frames and stack a subset. The masks are tied to the grid and to each frame's shift, so new `kcwi_xalign` shifts force a recompute. `crr_premask=True` adds `<base>.premsk.fits` voxels to the exclusion.

Step 5 — Check the stacks and hand them to GIST (optional)

`kcwi_xalign wcs` checks the two stacks against each other and only then writes the `_icubes_wcs.fits` files, the final cubes (and the ones GIST reads).

```
python kcwi_xalign.py wcs <t>.list <t>-red.list --z <z> [--ref image.fits] [--overwrite]
```

- It warns if the blue and red stacks are not on one pixel grid (more than 0.02 px apart), i.e. the `.par` files disagree.

- It measures red against blue on the stacks and warns above 0.15 px; with `--ref` it also reports blue against the reference.
- It copies `<t>_icubes.fits` and `<t>-red_icubes.fits` to `_icubes_wcs.fits` with a HISTORY line; the WCS itself is already right from the shifts. An existing file is kept unless `--overwrite` (with a backup).
- `--check-only` writes nothing; `--stack-suffix _wcs --check-only` measures an older `_wcs` pair.

Hand-off to GIST (optional, only for pPXF-fitted maps). Copy ten files into `<target>/gist-KCWI/inputData/`: `_icubes_wcs`, `_vcubes`, `_mcubes`, `_ecubes` and `_icubes` for each channel. Set `INPUT ./inputData/<t>` with `READ_DATA METHOD KECK_KCWIbpr`.

The reader then checks the hand-off itself:

- blue and red must agree on the sky to 0.05 px, or it stops;
- it warns when less than 15% of the grid has data and the data touch the edge (a cut-off stack, see below);
- it logs the dichroic gap to mask, and drops spaxels more than 20% masked in either channel.

The GIST side is in the separate GIST notes.

Known pitfalls

Most failures so far came from the grid and the absolute offset disagreeing, or from blue and red being registered separately.

Symptom	Cause	Fix
Red ends up ~0.18″ from blue	Using the red channel's own `astrom.list`	Use the blue `astrom.list` for both
Outer bins have true S/N ~4–7 while GIST says 35	Red sky residuals in the OH forest add up coherently over a bin; drizzled spaxels are correlated	S/N window on clean continuum (e.g. 6400–6800 Å rest), `COVARIANCE` ~0.5; `gist_snrcheck.py`

Two smaller effects: colour gradients in a galaxy give apparent blue–red offsets of about 0.02″, so keep the white-light windows next to the dichroic; and at low S/N the true alignment error is about twice the formal one.

Quick reference: a new target

In the stack folder, after the DRP, the saturation screen and red sky subtraction. `T` is the target name, `Z` its redshift.

```
# 0. lists and .par files: same dimension / xpix / ypix / orientation /
stack_ad in both
#    T.list, T-red.list, T.par, T-red.par

# 1. shifts for both channels, with the absolute position
python kcwi_xalign.py shifts T.list T-red.list --z Z --ref image.fits   # or
--astrom T.astrom.list

# 2. optional: compare with older shift lists
python kcwi_shiftcheck.py T.list T-red.list --a .shift.list.bak_<time> --b
.shift.list --z Z --plot check.png

# 3. stack both channels
python -c "from kcwikit.kcwi.kcwi import kcwi_stack; \
[kcwi_stack(l, crr_split=True, crr_split_mode='frame', crr_save_mask=True,
overwrite=True) \
 for l in ('T.list', 'T-red.list')]"

# 4. check blue vs red and write the _icubes_wcs files
python kcwi_xalign.py wcs T.list T-red.list --z Z --overwrite

# 5. optional, for pPXF maps: copy the 10 cubes to <target>/gist-
KCWI/inputData/ and continue with GIST
```

Before using the cubes, check:

- [ ] `wcs` reports one pixel grid and a red–blue residual under 0.15 px
- [ ] a mid-band plane has data over the galaxy, not in one corner
- [ ] `crr_split_mode='frame'` used when there are 7 or fewer frames